Self-Supervised Learning

Self-supervised learning (SSL) is a type of machine learning where:

- a model learns useful representations from **unlabelled data**
- by solving automatically generated (or "pretext") tasks.

Unlike supervised learning, which requires large amounts of labelled data, SSL leverages the structure of the input data itself to create labels or signals for training.

How it Works

1. **Pretext Task Creation**
   - The model is given an artificial problem that doesn't require human-labelled data.
   - Examples: predicting missing parts of data, reordering shuffled input, or contrasting similar vs. dissimilar data.
2. **Representation Learning**
   - By solving the pretext task, the model learns **general features/representations** about the data.
3. **Downstream Tasks**
   - The learned features are then fine-tuned or directly used for real tasks (classification, detection, recommendation, etc.).

🔹 Examples of Self-Supervised Learning

- **Natural Language Processing (NLP):**
  - Models like GPT, BERT, and LLaMA use SSL.
  - Pretext tasks: predict the next word (language modeling), fill in masked words (masked language modeling).
- **Computer Vision:**
  - Contrastive learning (e.g., SimCLR, MoCo): learn embeddings where augmented versions of the same image are close, and different images are far apart.
  - Image inpainting or jigsaw puzzles: predict missing parts or reorder shuffled patches.
- **Speech & Audio:**
  - Predict masked parts of audio (e.g., wav2vec).

🔹 How It's Used

1. **Pretraining Large Models:**
   - SSL is the foundation of modern large language models and vision transformers.
   - It allows training on massive unlabeled datasets (text, images, audio).

2. **Transfer Learning:**
   - The representations learned are transferred to smaller, task-specific datasets where labels exist.
3. **Real-World Applications:**
   - Chatbots & assistants (like me 🙂).
   - Image recognition without massive labeled datasets.
   - Speech recognition with less transcribed data.
   - Medical imaging (where labeling is expensive).

---

1) The idea / signal: create a *pretext task*

What: Invent a task where the training signal comes from the data itself (no human labels).

- **CV example — contrastive view task (SimCLR style):**

  Take an image, make two random augmented views (crop, color-jitter, flip). Train the model to *pull* embeddings of these two views together and *push* embeddings of other images apart.

  Concrete: original image → two crops → model produces embeddings z_1, z_2 → objective encourages $z_1 \approx z_2$.
- **NLP example — Masked Language Modeling (BERT):**

  Sentence: "The cat sat on the mat." Mask a token: "The cat [MASK] on the mat." Train the model to predict the missing word ("sat").

2) Data augmentations & positive/negative construction

Why: For SSL you usually need pairs (or corrupted inputs) that define what should be similar/different.

- **CV augments:** random resized crop, color jitter, Gaussian blur, horizontal flip, solarize.

  Example positive pair: two different crops of the same photo. Negatives: other images in the batch.

- **NLP "augments":** token masking, token replacement, sentence shuffling, back-translation, span corruption.

  Example: mask 15% tokens randomly (BERT); or corrupt spans and ask the model to reconstruct (T5 / span corruption).

3) The encoder + projector architecture

What: A backbone encoder converts input → representation. Often a small MLP "projector" maps representation → space where SSL loss is applied.

- **CV:** encoder = ResNet / ViT → global feature vector. Projector head (2–3 layer MLP) → contrastive embedding. At evaluation you typically *drop* the projector and use encoder features.
- **NLP:** encoder = Transformer (BERT: encoder stack; GPT: decoder stack). For MLM you use token-level prediction heads; for autoregressive LM you directly predict next-token logits.

4) Objective / loss functions (math)

Short, important formulas you'll see.

- **Contrastive (InfoNCE)** — used in SimCLR / MoCo: for anchor i and positive j:

```
L_i = - log ( exp(sim(z_i, z_j)/τ) / Σ_k exp(sim(z_i, z_k)/τ) )
```

where `sim` is cosine similarity and `τ` is temperature; sum over k runs over positives+negatives (often the batch).

- **Masked LM (BERT)**:

```
L = - Σ_{positions m masked} log p(token_true_m | context_with_mask)
```

(i.e. cross-entropy only at masked positions)

- **Autoregressive LM (GPT)**:

```
L = - Σ_t log p(x_t | x_{<t})
```

- **BYOL / non-contrastive (sketch)**: learn by regression between two augmented views using a target network and stop-gradient. No explicit negatives; stability techniques (stop-grad, momentum encoder) are used.
- **Replaced token detection (ELECTRA):** classifier predicts whether each token was replaced by a generator; loss is binary cross-entropy per token.

5) Training procedure: pretrain → probe/finetune

Process:

1. **Pretrain** encoder on large unlabeled dataset with SSL objective.
   - CV: millions of images with augmentations.
   - NLP: huge text corpora (web, books, Wikipedia).
2. **Evaluate** learned features: common pattern is **linear probe —** freeze encoder, train a simple classifier on a small labeled set to test representation quality.
3. **Fine-tune** entire model on downstream labeled task (e.g., image classification, question answering).

Examples:

- Pretrain SimCLR on ImageNet without labels → freeze encoder and train linear classifier on ImageNet labels (linear evaluation).
- Pretrain BERT on Wikipedia+Books with MLM → fine-tune on SST-2 sentiment dataset or SQuAD QA.

6) Downstream tasks & evaluation

CV downstreams: image classification, object detection, instance segmentation, retrieval. Metrics: accuracy, mAP, IoU.

NLP downstreams: classification (accuracy/F1), named entity recognition (F1), QA (EM / F1), generation (BLEU/ROUGE/ROUGE-L/ROUGE-2 / human eval).

7) Practical tricks and why they matter

- **Batch size / negatives:** Contrastive methods benefit from many negatives (large batch or memory queue like MoCo).
- **Temperature τ:** controls sharpness in InfoNCE; tuning matters.
- **Projector head:** discard at eval — the projector improves training dynamics.
- **Momentum encoder / queue (MoCo):** stabilizes negative representations.
- **Stop-gradient (BYOL):** prevents collapse without negatives.
- **Masking ratio (BERT / MAE):** e.g., BERT uses ~15% token masking; MAE (vision) masks large patch proportions (~75%) — architecture and decoder choices matter.
- **Compute & data:** scale helps — larger models + more data give better general features.
- **Data domain match:** pretrain on data similar to your downstream domain if possible.

8) Variants — quick tour (so you know the names)

- **Contrastive:** SimCLR, MoCo — explicit negatives, InfoNCE.
- **Non-contrastive / bootstrap:** BYOL, SimSiam, DINO — no explicit negatives.
- **Masked modeling:** BERT (text), MAE (vision) — mask & reconstruct.
- **Replaced token detection:** ELECTRA — discriminative replaced-token objective.
- **Generative seq-to-seq:** T5 — corrupt text and learn to generate original.

9) Tiny pseudo-examples (conceptual code)

CV — SimCLR training step (sketch):

```
# x = image batch x1 = augment(x) x2 = augment(x) h1 = encoder(x1); z1 = p
rojector(h1) h2 = encoder(x2); z2 = projector(h2) loss = InfoNCE(z1, z2, n
egatives_in_batch) # backprop loss → update encoder + projector
```

NLP — Masked LM step (sketch):

```
# tokens = [The, cat, sat, on, the, mat] masked_tokens, labels = mask_some
_tokens(tokens) # e.g., replace with [MASK] outputs = transformer(masked_t
okens) loss = cross_entropy(outputs_at_mask_positions, labels) # backprop
→ update transformer
```

10) How to pick a method for a project

- Need token-level understanding (NLP): **masked LM (BERT)** or ELECTRA.
- Want generative output (summarization, story): **autoregressive pretraining (GPT)** or seq2seq (T5).
- Image tasks where labels are scarce: **contrastive (SimCLR/MoCo)** or **masked-image modeling (MAE)**.
- Limited compute and want simplicity: masked methods (MAE/BERT) can be more sample/economical than big contrastive setups.

11) Small experiment suggestions (hands-on)

- **CV toy:** take CIFAR-10 (unlabeled) → implement SimCLR with small ResNet, do linear probe on CIFAR-10 labels.
- **NLP toy:** take small Wikipedia dump → train a small BERT (tiny) with MLM, fine-tune on SST-2 for sentiment.

DAEs — Denoising Autoencoders

Context Encoders

CPC — Contrastive Predictive Coding

MoCo — Momentum Contrast

CLIP — Contrastive Language–Image Pretraining

SimCLR